

# Deploying Solaris Health

---

Solaris is a two-part app: a **Node/Express backend** ( `backend/` ) and a **Vite/React frontend** (repo root: `src/` , `public/` , `index.html` ). It is 12-factor and portable — every secret comes from an environment variable and the database is addressed solely by `DATABASE_URL` , so the same code runs on the current VM, Railway, Render, Heroku, or any container host with **zero code changes**.

This repo ships ready-to-use configs:

| File                                                                                                 | Platform                                                         |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <code>docker-compose.yml</code> + <code>Dockerfile.backend</code> / <code>Dockerfile.frontend</code> | Current VM (Docker)                                              |
| <code>railway.json</code>                                                                            | Railway (Dockerfile build)                                       |
| <code>render.yaml</code>                                                                             | Render (blueprint: backend + static frontend + managed Postgres) |
| <code>Procfile</code>                                                                                | Heroku / Dokku / any Procfile runner                             |

---

## Required environment variables

---

Never commit real values. Copy the examples and fill them in on the platform's secret store:

- Backend: `backend/.env.example` (backend/.env.example)
- Frontend build: `VITE_API_URL`

| Variable                                                                                  | Where          | Notes                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code>                                                                 | backend        | <b>Single source of truth</b> for the DB. <code>postgresql://user:pass@host:5432/db</code> . Add <code>?sslmode=require</code> for managed Postgres (Neon/Render/RDS) — the pool enables SSL automatically when it sees that. |
| <code>JWT_SECRET</code>                                                                   | backend        | 64-byte hex. Generate: <code>node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"</code> . The server <b>refuses to boot</b> without it.                                                                  |
| <code>PORT</code>                                                                         | backend        | Defaults to <code>5000</code> . Railway/Render/Heroku inject their own — the app honours <code>process.env.PORT</code> .                                                                                                      |
| <code>NODE_ENV</code>                                                                     | backend        | <code>production</code> .                                                                                                                                                                                                     |
| <code>ALLOWED_ORIGINS</code>                                                              | backend        | Comma-separated CORS allowlist, e.g. <code>https://your-domain,http://localhost:3000</code> .                                                                                                                                 |
| <code>LUCA_AI_MODE</code>                                                                 | backend        | <code>mock</code>   <code>cloud</code>   <code>anthropic</code>   <code>local</code> . Defaults to <code>mock</code> (works with no keys).                                                                                    |
| <code>LUCA_AI_BASE_URL</code> / <code>LUCA_AI_MODEL</code> / <code>LUCA_AI_API_KEY</code> | backend        | Needed when <code>LUCA_AI_MODE=cloud</code> .                                                                                                                                                                                 |
| <code>PG_POOL_MAX</code>                                                                  | backend        | Optional. Max pool connections (default <code>10</code> ). Keep at/under your provider's connection ceiling.                                                                                                                  |
| <code>PGSSL</code>                                                                        | backend        | Optional. <code>true</code> forces SSL when the URL has no <code>sslmode=require</code> .                                                                                                                                     |
| <code>VITE_API_URL</code>                                                                 | frontend build |                                                                                                                                                                                                                               |

| Variable | Where | Notes                                                    |
|----------|-------|----------------------------------------------------------|
|          |       | Public backend URL, baked into the bundle at build time. |

## Database migrations

Migrations live in `backend/migrations/` (plain SQL, run by **node-pg-migrate**). They are **idempotent** and tracked in a `pgmigrations` table, so running them repeatedly is safe.

The backend **runs pending migrations automatically on startup** (see the bottom of `backend/src/server.js`). To run them manually:

```
cd backend
DATABASE_URL=postgresql://... npm run migrate           # apply pending
DATABASE_URL=postgresql://... npm run migrate:status    # dry-run: show what would ap-
ply
DATABASE_URL=postgresql://... npm run migrate:down      # roll back the last migra-
tion
```

`render.yaml` and `Procfile` also invoke `npm run migrate` before `node src/server.js`.

## Seed / reset demo data

Demo accounts (all password `demo123`): `sofia@solaris.health`, `alejandro@solaris.health`, `admin@solaris.health`.

```
cd backend
DATABASE_URL=postgresql://... npm run seed             # seed demo data (idempotent)
DATABASE_URL=postgresql://... npm run seed:reset       # wipe + reseed demo data
```

## Platform: current VM (Docker Compose)

```
cd /home/ubuntu/luca-passport
docker compose up -d --build
curl -s https://solaris-health.abacusai.cloud/api/health # {"status":"ok",...}
node backend/scripts/smoke-test.js                      # 10/10 expected
```

The Postgres container holds the live data; `Dockerfile.backend` uses `dumb-init` as PID 1 so `docker stop` / redeploy triggers a graceful drain (see Graceful shutdown below).

## Platform: Railway

1. New Project → Deploy from Repo. Railway reads `railway.json` and builds `Dockerfile.backend`.
2. Add a **Postgres** plugin; Railway sets `DATABASE_URL`.

3. Add the backend env vars above ( `JWT_SECRET` , `LUCA_AI_*` , `ALLOWED_ORIGINS` , `NODE_ENV` ).
4. Healthcheck is preconfigured to `/api/health` . Migrations run on boot.
5. Deploy the frontend separately (static site) with `VITE_API_URL` set to the backend URL.

## Platform: Render

---

1. New → **Blueprint**, point at this repo. Render reads `render.yaml` and provisions:
  - `solaris-backend` (Node web service, `rootDir: backend` , health check `/api/health` )
  - `solaris-frontend` (static site built from repo root to `./dist` )
  - `solaris-db` (managed Postgres; `DATABASE_URL` auto-wired, includes `sslmode=require` )
2. Fill the `sync: false` secrets in the dashboard: `LUCA_AI_BASE_URL` , `LUCA_AI_API_KEY` , `ALLOWED_ORIGINS` , `VITE_API_URL` . `JWT_SECRET` is auto-generated.
3. `startCommand` runs `npm run migrate && node src/server.js` .

## Platform: Heroku (Procfile)

---

1. `heroku create` + add the **Heroku Postgres** add-on ( `DATABASE_URL` auto-set, SSL on).
2. `heroku config:set JWT_SECRET=... NODE_ENV=production LUCA_AI_MODE=cloud LUCA_AI_BASE_URL=... LUCA_AI_API_KEY=... ALLOWED_ORIGINS=...`
3. `git push heroku` . The `Procfile` runs migrations then starts the server.
4. Host the frontend build ( `npm run build` → `dist/` ) on any static host with `VITE_API_URL` set.

## Graceful shutdown

---

`backend/src/server.js` handles `SIGTERM` / `SIGINT` : it stops accepting new connections, drains in-flight requests, closes the Postgres pool, and exits — with a 10s forced-exit safety net. `dumb-init` in `Dockerfile.backend` ensures the signal actually reaches Node.

## Migrating the live database to managed Postgres

---

See [NEON\\_MIGRATION.md](#) (`NEON_MIGRATION.md`) for a step-by-step `pg_dump` → Neon → cutover.